

University of Tehran
College of Engineering
School of Electrical and Computer
Engineering

Computer Networks

Programming Project 02

Wireless Local Area Network Performance Evaluation (Wi-Fi 5 & 6)

Team Members

Shayan Maleki

Student ID: 810102515

Mani Hosseini

Student ID: 810102552

May 26, 2026

Table of Contents

1	Architecture and Design	3
1.1	Phase 1	3
1.2	Phase 2	3
1.3	Phase 3	3
2	Protocols and Parameters	4
2.1	Phase 1	4
2.2	Phase 2	4
2.3	Phase 3	5
3	Concept Explanations	6
3.1	Phase 1 Concepts	6
3.2	Phase 2 Concepts	6
3.3	Phase 3 Concepts	7
4	Execution and Screenshots	8
4.1	Phase 1	8
4.2	Phase 2	8
4.3	Phase 3	8
5	Results Analysis, Tables, and Charts	16
5.1	Phase 1 vs Phase 2 Baseline Comparison	16
5.2	Phase 1: Impact of Noise and Distance	16
5.3	Phase 2: Impact of Network Density	17
5.4	Phase 3: Impact of Extreme Congestion	17
6	Answers to Embedded Questions	18
6.1	Phase 1	18
6.1.1	What happens to the network when you increase the noise parameter?	18
6.2	Phase 2	18
6.2.1	What is the difference between DataMode and ControlMode and what role do MCS rates play in them?	18
6.2.2	Why did the network performance change between phase 1 and phase 2?	18
6.3	Phase 3	18
6.3.1	How does distance impact SINR and throughput?	18

6.3.2	What does the nRaRus parameter determine and what happens when you change it?	19
6.3.3	Explain what UseCentral26TonesRus does.	19
6.3.4	Provide a clear BSRP scheduling example.	19
6.3.5	Does a random start time decrease or increase collisions and why?	19
7	Challenges and Problems	20
7.1	Phase 1 Challenges	20
7.2	Phase 2 Challenges	20
7.3	Phase 3 Challenges	20
8	Phase 4: Reinforcement Learning Implementation	22
8.1	Introduction and Objective	22
8.2	System Architecture and IPC Bridge	22
8.3	Telemetry and the MacTx Fix	22
8.4	Reinforcement Learning Framework	22
8.5	Evaluation and Overfitting Analysis	23
8.6	Evaluation and TensorBoard Analysis	23
8.6.1	The Overfitting Vulnerability	25
8.7	Final Performance Comparison: AI vs. Standard UORA	25
8.7.1	The Overfitting Reality Check	26

Architecture and Design

1.1 Phase 1

In the first phase we made a simple baseline network using Wi-Fi 5 standard. The architecture is a static star topology. We put one AP in the center at 0,0,0 and 5 stations around it in a circle with 5 meters distance. We used ConstantPositionMobilityModel so nodes don't move during the 10 seconds simulation. The AP is setup as a UDP echo server and the 5 stations are UDP echo clients. This means traffic is two-way. The stations send data to the AP and the AP sends it right back. This lets us see how the basic CSMA/CA works with both uplink and downlink traffic in a simple network.

1.2 Phase 2

For phase 2 the physical topology is exactly the same as phase 1. We still have one AP and 5 stations in a 5 meter circle. The main difference is in the MAC and physical layer. We changed the standard to Wi-Fi 6 and added 4x4 MIMO antennas. The AP now uses OFDMA to schedule the traffic. The application layer still uses the two-way UDP echo setup so we can easily compare the results with phase 1 and see how Wi-Fi 6 OFDMA handles bidirectional traffic better than Wi-Fi 5.

1.3 Phase 3

Phase 3 is a lot more realistic and congested. We increased the number of stations to 40 nodes. They are not in a perfect circle anymore. We used RandomDiscPositionAllocator to put them in random distances between 5 and 30 meters from the AP. We also gave them height so the AP is at 3 meters and stations are at 1 meter. This makes the SINR different for each node. A major change here is the traffic. We removed the echo server and replaced it with a PacketSink. The stations just send UDP packets to the AP and the AP absorbs them without answering back. We made it strictly one-way uplink traffic to perfectly isolate and test the UORA mechanism. Also the traffic start time is random now so they don't all send packets at exactly 0.0s which would cause huge collisions.

Protocols and Parameters

2.1 Phase 1

In the first phase we used the YansWifiPhy layer for the physical setup. This is the basic physical layer in ns-3 and it is enough for simple Wi-Fi 5 simulations. It treats the whole channel as one big block.

```
YansWifiChannelHelper channel = YansWifiChannelHelper::Default();  
YansWifiPhyHelper physicalLayer;  
physicalLayer.SetChannel(channel.Create());
```

For the MAC layer we set the standard to 802.11ac. The setup is very simple here we just defined the SSID and installed ApWifiMac for the access point and StaWifiMac for the stations without any extra multi user controls.

```
WifiHelper wifi;  
wifi.SetStandard(WIFI_STANDARD_80211ac);
```

2.2 Phase 2

For phase 2 we had to completely change the physical layer to SpectrumWifiPhy. We did this because the old Yans physical layer cannot divide the frequency into small subchannels and Wi-Fi 6 OFDMA needs this to work. We also added 4x4 MIMO antennas and set the bandwidth to 40MHz.

```
SpectrumWifiPhyHelper phy;  
SpectrumChannelHelper channelHelper = SpectrumChannelHelper::Default();  
channelHelper.SetChannel("ns3::MultiModelSpectrumChannel");  
phy.SetChannel(channelHelper.Create());  
phy.Set("Antennas", UIntegerValue(4));  
phy.Set("ChannelSettings", StringValue("{38, 40, BAND_5GHZ, 0}"));
```

In the MAC layer we changed the standard to 802.11ax. The biggest change is that we gave the AP a multi user scheduler. We used RrMultiUserScheduler and enabled uplink OFDMA so the AP can manage multiple stations at the same time. We also enabled BSRP here even though it is officially a phase 3 topic. We enabled enableBsrp because it is a necessary supporting mechanism for the basic uplink OFDMA scheduler to function correctly. The AP needs it to know how much data stations have in their queue.

```
mac.SetMultiUserScheduler("ns3::RrMultiUserScheduler",  
                           "EnableU10fdma", BooleanValue(true),  
                           "EnableBsrp", BooleanValue(true),  
                           "NStations", UIntegerValue(8),  
                           "UseCentral26TonesRus", BooleanValue(false));
```

We also had to configure the MCS variables to control the modulation speed. We set the DataMode to HeMcs7 to send data packets as fast as possible. But we set ControlMode to HeMcs5 which is slower because control signals need more reliability to not drop.

```
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",  
                             "DataMode", StringValue("HeMcs7"),  
                             "ControlMode", StringValue("HeMcs5"));
```

2.3 Phase 3

In phase 3 we kept the Spectrum physical layer but we added transmission power limits. We forced the Tx power to 16 dBm. This is very important because it makes the signal degrade realistically over the random distances so we can measure the true SINR.

```
phy.Set("TxPowerStart", DoubleValue(16.0));  
phy.Set("TxPowerEnd", DoubleValue(16.0));
```

For the MAC layer we used the same multi user scheduler but we set UseCentral26TonesRus to true. This gives the AP an extra small RU in the center of the frequency which is very useful for sending the tiny BSRP trigger frames without wasting big data RUs.

```
mac.SetMultiUserScheduler("ns3::RrMultiUserScheduler",  
                           "EnableU10fdma", BooleanValue(true),  
                           "EnableBsrp", BooleanValue(true),  
                           "UseCentral26TonesRus", BooleanValue(true),  
                           "NStations", UIntegerValue(8));
```

Finally we changed the MCS rates for this phase. Because the network has 40 nodes and heavy congestion we lowered the DataMode to HeMcs5 and ControlMode to HeMcs2. We did this because lower MCS rates are much more resistant to noise and interference in a crowded UORA network.

Concept Explanations

3.1 Phase 1 Concepts

In phase 1 the main concept we need to explain is Jain's Fairness Index. It is a math formula that tells us if the network is fair and if all stations are getting an equal share of the bandwidth.

The formula is calculated like this:

$$J = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2}$$

In this formula x_i is the throughput of each station and n is the total number of stations. The answer is always a number between 0 and 1. If the result is close to 1 it means the AP is treating everyone equally and the network is very fair. If it is close to 0 it means one station is hogging all the channel and the others are starving.

3.2 Phase 2 Concepts

In phase 2 we moved to Wi-Fi 6 so we have to explain three major new technologies.

First is OFDMA. In old Wi-Fi like phase 1 only one device could use the channel at a time. OFDMA fixes this by dividing the main 40MHz frequency bandwidth into much smaller sub-channels called Resource Units or RUs. Because the channel is chopped into smaller pieces the AP can talk to multiple stations at the exact same time by giving each station a different RU. This is extremely useful when stations are sending small packets because they don't waste the whole channel.

Second is MU-MIMO. This also lets the AP talk to multiple users at the same time but it does it completely differently than OFDMA. OFDMA divides the frequency but MU-MIMO divides the space. It uses our 4x4 antennas to beam different data streams to different users on the exact same frequency at the exact same time. So OFDMA is frequency division and MU-MIMO is spatial division.

Third is MCS which stands for Modulation and Coding Scheme. MCS basically decides how fast and how strong the Wi-Fi signal is. A high MCS rate like HeMcs7 packs a lot of data into the wave so it is very fast but it is fragile. If there is a little bit of noise the packet will break and drop. A low MCS rate like HeMcs5 or HeMcs2 is much slower but it adds extra coding to make the signal super strong and reliable against noise. This is exactly why we set DataMode to a high MCS to get high throughput but we set ControlMode to a low MCS. Control frames manage the network so if they drop the whole connection breaks. They must have maximum reliability.

3.3 Phase 3 Concepts

For the last phase we need to explain how Wi-Fi 6 handles heavy congestion using BSRP, UORA and MU-EDCA.

First is BSRP which stands for Buffer Status Report Polling. Because the AP schedules all the traffic in Wi-Fi 6 it needs to know how much data each station has in its queue. It can't just guess. So the AP sends a special control frame called a BSRP trigger frame. The stations hear this and reply with a report telling the AP exactly how many packets they have ready to send. Then the AP can schedule the exact right size Resource Units for them.

Second is UORA which stands for Uplink OFDMA Random Access. In phase 2 the OFDMA was perfectly scheduled by the AP. But in a real network with 40 nodes sometimes stations need to send small data or buffer reports without waiting for a specific invitation. So the AP leaves some of the OFDMA subchannels completely open for anyone to use. These are called Random Access Resource Units or RA-RUs. If all 40 stations try to talk on the open RA-RUs at the exact same time they will collide. To fix this UORA uses a backoff timer. Instead of shouting immediately each station picks a random number and counts down. When their counter hits zero they finally transmit. This spreads the traffic out and stops massive collisions.

Third is MU-EDCA which stands for Multi-User Enhanced Distributed Channel Access. This is a fairness mechanism. In Wi-Fi 6 stations can send data in two ways they can be scheduled by the AP or they can fight for the channel randomly. If a station gets lucky and the AP gives it a scheduled time to send data it should not be allowed to immediately fight for the random channel right after because that is greedy and unfair to other stations. So MU-EDCA gives a penalty to stations that just used a scheduled OFDMA transmission. It forces their random access timer to be much longer. This stops the scheduled stations from hogging the channel and keeps the network fair for everyone.

Execution and Screenshots

4.1 Phase 1

For the first phase we ran the simulation three times in the terminal to test how noise and distance affect Wi-Fi 5. First we ran the normal baseline with low noise at 5 meters distance. Then we added high noise but kept the 5 meter distance. Finally we kept the high noise but moved the stations to 25 meters away to see the real signal drop.

4.2 Phase 2

In phase 2 we ran the code three times to test how OFDMA handles different network densities. We ran it with 2 stations then 5 stations for our baseline comparison and finally 15 stations to crowd the channel.

4.3 Phase 3

For phase 3 we ran the heavy UORA simulation with 40 nodes. We did two tests by changing the packet interval to flood the network. First we used a 5ms interval and then a completely flooded 1ms interval.

```
Throughput: 16.8309 kbps
Avg Delay: 1.28711 ms

Flow: 192.168.1.3 -> 192.168.1.1
  Sent: 20 packets
  Received: 19 packets
  Lost: 1 (5%)
  Throughput: 8.63912 kbps
  Avg Delay: 1.35378 ms

Flow: 192.168.1.4 -> 192.168.1.1
  Sent: 20 packets
  Received: 19 packets
  Lost: 1 (5%)
  Throughput: 16.83 kbps
  Avg Delay: 1.94202 ms

Flow: 192.168.1.5 -> 192.168.1.1
  Sent: 20 packets
  Received: 19 packets
  Lost: 1 (5%)
  Throughput: 8.6397 kbps
  Avg Delay: 1.32048 ms

Flow: 192.168.1.6 -> 192.168.1.1
  Sent: 20 packets
  Received: 19 packets
  Lost: 1 (5%)
  Throughput: 16.8305 kbps
  Avg Delay: 1.36811 ms

-----
SUMMARY
-----
Number of upload flows: 5
Average Throughput: 13.554 kbps
Average Delay: 1.4543 ms
Jain's Fairness Index: 0.919413
```

Figure 1: Phase 1 - Low Noise Baseline (5m)

```
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\U
Throughput: 16.8305 kbps
Avg Delay: 1.35074 ms

Flow: 192.168.1.3 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 8.63909 kbps
Avg Delay: 1.37131 ms

Flow: 192.168.1.4 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 16.8296 kbps
Avg Delay: 1.78347 ms

Flow: 192.168.1.5 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 8.6397 kbps
Avg Delay: 1.27078 ms

Flow: 192.168.1.6 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 16.8308 kbps
Avg Delay: 1.45381 ms

-----
SUMMARY
-----
Number of upload flows: 5
Average Throughput: 13.5539 kbps
Average Delay: 1.44603 ms
Jain's Fairness Index: 0.919415
```

Figure 2: Phase 1 - High Noise (5m)

```
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Ubuntu
Throughput: 16.8277 kbps
Avg Delay: 3.14404 ms

Flow: 192.168.1.3 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 8.63751 kbps
Avg Delay: 2.66167 ms

Flow: 192.168.1.4 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 16.8295 kbps
Avg Delay: 2.9903 ms

Flow: 192.168.1.5 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 8.63943 kbps
Avg Delay: 2.03029 ms

Flow: 192.168.1.6 -> 192.168.1.1
Sent: 20 packets
Received: 19 packets
Lost: 1 (5%)
Throughput: 16.8245 kbps
Avg Delay: 2.91141 ms

-----
SUMMARY
-----
Number of upload flows: 5
Average Throughput: 13.5517 kbps
Average Delay: 2.74754 ms
Jain's Fairness Index: 0.919429
```

Figure 3: Phase 1 - High Noise with Long Distance (25m)

```
Wi-Fi 6 (802.11ax) - Phase 2
OFDMA + MU-MIMO

Stations: 2 | AP: 1
Installed Wi-Fi 6 (802.11ax) - 40 MHz channel
2 stations in a 5m circle around AP
AP IP: 192.168.1.1
STA IP range: 192.168.1.2 - 192.168.1.3
Traffic: 2 clients → AP, 500ms interval, 1024/512B packets

Running simulation for 10 seconds...

=====
RESULTS - Wi-Fi 6 (802.11ax) OFDMA + MU-MIMO
=====

Flow 1: 192.168.1.2 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 15.9452 kbps Delay: 1.21558 ms

Flow 2: 192.168.1.3 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 8.18434 kbps Delay: 1.3252 ms

=====
SUMMARY
=====

Flows: 2
Total Tx: 40 Rx: 36 Lost: 4 (10%)
Average Throughput: 12.0648 kbps
Total Throughput: 24.1296 kbps
Average Delay: 1.27039 ms
Jain's Fairness Index: 0.90625
```

Figure 4: Phase 2 - OFDMA with 2 Stations

RESULTS - Wi-Fi 6 (802.11ax) OFDMA + MU-MIMO

Flow 1: 192.168.1.2 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 15.9446 kbps Delay: 1.24907 ms

Flow 2: 192.168.1.3 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 8.18502 kbps Delay: 1.30773 ms

Flow 3: 192.168.1.4 → 192.168.1.1
Tx: 20 Rx: 17 Lost: 3 (15%)
Throughput: 15.0588 kbps Delay: 1.74393 ms

Flow 4: 192.168.1.5 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 8.1845 kbps Delay: 1.29919 ms

Flow 5: 192.168.1.6 → 192.168.1.1
Tx: 20 Rx: 19 Lost: 1 (5%)
Throughput: 16.8304 kbps Delay: 1.30508 ms

SUMMARY

Flows: 5
Total Tx: 100 Rx: 90 Lost: 10 (10%)
Average Throughput: 12.8407 kbps
Total Throughput: 64.2033 kbps
Average Delay: 1.381 ms
Jain's Fairness Index: 0.917809

Figure 5: Phase 2 - OFDMA Baseline (5 Stations)

```

Flow 9: 192.168.1.10 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 15.9439 kbps Delay: 2.7548 ms

Flow 10: 192.168.1.11 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 8.18295 kbps Delay: 3.32497 ms

Flow 11: 192.168.1.12 → 192.168.1.1
Tx: 20 Rx: 17 Lost: 3 (15%)
Throughput: 15.8947 kbps Delay: 3.10305 ms

Flow 12: 192.168.1.13 → 192.168.1.1
Tx: 20 Rx: 17 Lost: 3 (15%)
Throughput: 7.72947 kbps Delay: 3.31966 ms

Flow 13: 192.168.1.14 → 192.168.1.1
Tx: 20 Rx: 19 Lost: 1 (5%)
Throughput: 16.8297 kbps Delay: 2.86107 ms

Flow 14: 192.168.1.15 → 192.168.1.1
Tx: 20 Rx: 18 Lost: 2 (10%)
Throughput: 8.18213 kbps Delay: 4.29884 ms

Flow 15: 192.168.1.16 → 192.168.1.1
Tx: 20 Rx: 17 Lost: 3 (15%)
Throughput: 15.0582 kbps Delay: 2.12755 ms

=====
SUMMARY
=====
Flows: 15
Total Tx: 300 Rx: 268 Lost: 32 (10.6667%)
Average Throughput: 12.4019 kbps
Total Throughput: 186.028 kbps
Average Delay: 3.12688 ms
Jain's Fairness Index: 0.908746

```

Figure 6: Phase 2 - OFDMA Crowded (15 Stations)

```

=====
SUMMARY
=====
Flows: 40
Total Tx: 79822 | Total Rx: 68400 | Lost: 11422 (14.3093%)
Average Throughput: 166.458 kbps
Total Throughput: 6658.31 kbps
Average Delay: 29.2518 ms
Jain's Fairness Index: 0.946121

```

Figure 7: Phase 3 - UORA with 5ms Interval

```
=====
SUMMARY
=====
Flows: 40
Total Tx: 399047 | Total Rx: 206918 | Lost: 192129 (48.147%)
Average Throughput: 640.338 kbps
Total Throughput: 25613.5 kbps
Average Delay: 82.7418 ms
Jain's Fairness Index: 0.78558
```

Figure 8: Phase 3 - UORA Flooded with 1ms Interval

Results Analysis, Tables, and Charts

5.1 Phase 1 vs Phase 2 Baseline Comparison

First we need to compare Wi-Fi 5 against Wi-Fi 6 directly. To do this we look at the 5 station baseline for both phases.

Scenario (5 Stations)	Avg Throughput	Avg Delay	Packet Loss
Phase 1 (Wi-Fi 5)	13.55 kbps	1.45 ms	5%
Phase 2 (Wi-Fi 6 OFDMA)	12.84 kbps	1.38 ms	10%

Table 1: Baseline Comparison

When we look at the table we can see that for a small network of 5 stations upgrading to Wi-Fi 6 didnt magically fix everything. The average delay actually got a tiny bit better dropping from 1.45ms to 1.38ms because OFDMA schedules things nicely. But the packet loss doubled to 10 percent. This happens because OFDMA requires extra control frames to manage the RUs and in a mostly empty network this just adds unnecessary overhead. This proves OFDMA is not meant for empty networks.

5.2 Phase 1: Impact of Noise and Distance

Next we analyze how Wi-Fi 5 handles bad environments.

Phase 1 Test	Avg Throughput	Avg Delay	Packet Loss
Low Noise (5 meters)	13.55 kbps	1.45 ms	5%
High Noise (5 meters)	13.55 kbps	1.44 ms	5%
High Noise (25 meters)	13.55 kbps	2.74 ms	5%

Table 2: Noise and Distance Analysis

The data shows exactly how path loss works. When we just added high noise at 5 meters nothing changed at all. The delay stayed at 1.44ms. This is because at 5 meters the signal is so strong it completely ignores the background noise. But when we moved the stations to 25 meters the signal became weak. Then the high noise actually damaged the SINR and forced the delay to almost double to 2.74ms because the network struggled to process the weak signals.

5.3 Phase 2: Impact of Network Density

Here we prove why OFDMA exists by changing the number of stations.

Number of Stations	Total Throughput	Avg Delay	Packet Loss
2 Stations	24.12 kbps	1.27 ms	10.0%
5 Stations	64.20 kbps	1.38 ms	10.0%
15 Stations	186.02 kbps	3.12 ms	10.6%

Table 3: OFDMA Density Analysis

This is the most important result for phase 2. When we only had 2 stations the total network throughput was very low at 24 kbps. But when we crowded the network with 15 stations the total throughput exploded to 186 kbps. The delay went up to 3.12ms because the AP has to do a lot more math to schedule 15 different nodes into subchannels but the packet loss barely moved at all staying around 10.6 percent. OFDMA packed them all in and handled the crowd perfectly without crashing.

5.4 Phase 3: Impact of Extreme Congestion

Finally we look at the random access UORA network with 40 stations under heavy load.

Packet Interval	Total Throughput	Avg Delay	Packet Loss	Fairness
5ms Interval	6658 kbps	29.2 ms	14.3%	0.94
1ms Interval	25613 kbps	82.7 ms	48.1%	0.78

Table 4: UORA Congestion Analysis

When the interval was 5ms the UORA backoff timer managed the 40 nodes pretty well. The fairness was amazing at 0.94. But when we changed the interval to 1ms we completely flooded the channel. All 40 nodes tried to shout at the same time. The total throughput went up to 25 Mbps because the channel was completely full but the network suffered hard. Packet loss spiked to 48 percent and delay jumped to 82ms. Also the fairness dropped to 0.78 because the random stations that spawned closer to the AP had a stronger SINR so they bullied the weak far away stations out of the random access RUs.

Answers to Embedded Questions

6.1 Phase 1

6.1.1 What happens to the network when you increase the noise parameter?

When we increase the noise parameter nothing actually happens if the stations are very close to the AP. Our 5 meter test showed no change at all because the signal was too strong. But when we moved the stations to 25 meters the signal suffered from natural path loss. At that distance the high noise completely crushed the SINR causing the average delay to almost double from 1.45ms to 2.74ms. So noise only destroys the network if the distance is already weakening the signal.

6.2 Phase 2

6.2.1 What is the difference between DataMode and ControlMode and what role do MCS rates play in them?

DataMode is the setting for sending the actual user payload packets like our UDP traffic. ControlMode is for the network management frames like ACKs and RTS/CTS. We set DataMode to a high MCS like HeMcs7 because we want the data to transfer as fast as possible. But we set ControlMode to a low MCS like HeMcs5 because control frames must have high reliability. If control frames drop the whole connection breaks. Low MCS is much more resistant to noise.

6.2.2 Why did the network performance change between phase 1 and phase 2?

The performance change was actually surprising. When we tested both phases with a small 5 station network Wi-Fi 6 actually performed worse. The packet loss doubled to 10 percent because OFDMA requires extra control frames like BSRP to schedule everything and in an empty network this just causes unnecessary overhead. But when we scaled phase 2 up to 15 stations the performance exploded to 186 kbps throughput while keeping the loss steady. This proves Wi-Fi 6 is worse for small networks but incredibly powerful for crowded networks.

6.3 Phase 3

6.3.1 How does distance impact SINR and throughput?

Distance directly hurts SINR. Nodes close to the AP at 5 meters have high SINR and nodes at 30 meters have very weak SINR. We saw this perfectly in our 1ms flooded UORA test. The fairness index dropped heavily to 0.78 because the close stations with strong SINR completely

bullied the far stations out of the random access channel. The far stations are forced to drop to lower MCS rates to survive the noise which ruins their throughput.

6.3.2 What does the nRaRus parameter determine and what happens when you change it?

nRaRus stands for number of Random Access Resource Units. It determines how many sub-channels the AP leaves open for UORA random contention. However we investigated the ns-3 source code for version 3.38 and found that this parameter is actually hardcoded in the trigger frame constructor and is not exposed as a configurable attribute in the script. So we could not manually change it to plot a graph. Theoretically if we decrease it collisions would spike and if we increase it we would waste data bandwidth.

6.3.3 Explain what UseCentral26TonesRus does.

When we set this to true it activates a very small 26 tone Resource Unit right in the exact center of the 40MHz channel band. This is extremely useful because the AP can use this tiny chunk of frequency specifically to send small control frames like BSRP trigger frames without wasting the larger RUs that are needed for actual heavy data.

6.3.4 Provide a clear BSRP scheduling example.

First the AP broadcasts a BSRP trigger frame to all stations asking who has data to send. Station 1 and station 2 reply with a buffer status report telling the AP their queues are full. The AP reads these reports and then sends a basic trigger frame that says station 1 can use RU number 1 and station 2 can use RU number 2. Then both stations send their data perfectly on schedule without colliding.

6.3.5 Does a random start time decrease or increase collisions and why?

Random start time definitely decreases collisions. If all 40 stations wake up and start their applications at exactly 0.0 seconds they will all try to send ARP resolution requests at the exact same nanosecond. This causes a massive wave of collisions and some stations completely fail to connect. By giving them a random start time between 0.0 and 0.05 seconds we spread out the initial traffic so the UORA backoff counters have time to actually work and manage the congestion.

Challenges and Problems

7.1 Phase 1 Challenges

The first phase was very straightforward and we did not encounter many problems. We used a simple star topology with standard Wi-Fi 5 features. The YansWifiPhy layer worked perfectly fine here because it treats the whole channel as one big block and doesn't need to do any advanced frequency division.

7.2 Phase 2 Challenges

Phase 2 had a lot of heavy technical problems because of the jump to Wi-Fi 6.

First was the physical layer crash. We initially tried to keep using YansWifiPhy like in phase 1 but it completely crashed the ns-3 simulator when we turned on OFDMA. We realized Yans cannot handle dividing the frequency into smaller subchannels so we had to completely rewrite the physical layer to use SpectrumWifiPhy. We also realized the standard 20MHz bandwidth was not wide enough to efficiently divide into Resource Units for multiple stations so we increased the channel width to 40MHz.

Second was the MCS speed crash. We originally got greedy and tried to push the data mode to HeMcs11 which is the absolute maximum speed and dropped control mode to HeMcs2. This completely crashed the network. HeMcs11 requires an absolutely perfect SINR and causes massive packet loss if there is even slight interference. The disparity caused the scheduler to fail so we had to dial it back to more mild and stable settings using HeMcs7 for data and HeMcs5 for control.

Third was the scheduler allocation. Inside the RrMultiUserScheduler there is an NStations parameter. If you leave it default or set it wrong the scheduler messes up the Resource Unit allocations and trigger frames. We explicitly hardcoded it to 8 to ensure the scheduler had plenty of capacity to perfectly manage our active stations without glitching.

7.3 Phase 3 Challenges

The biggest challenge in phase 3 was the nRaRus parameter. The project document asked us to analyze what happens when we change the number of Random Access Resource Units. We spent hours trying to configure this in our simulation script. We eventually had to dig into the actual ns-3.38 C++ source code and found out that this parameter is hardcoded deep inside the trigger frame constructor. It is not actually exposed to the user as a configurable attribute so we couldn't easily plot a graph for it without modifying the core simulator files.

Another major issue was the initial 40 node congestion. When we first ran 40 stations the network immediately broke. If all 40 nodes wake up at exactly 0.0 seconds they all send ARP requests at the exact same nanosecond causing a massive collision that UORA cannot fix. We had to implement a random start time for the applications just to let the BSRP polling and UORA backoff counters initialize properly without instantly dropping all connections.

Phase 4: Reinforcement Learning Implementation

8.1 Introduction and Objective

For the final bonus phase we wanted to fix a major problem with standard UORA. The standard Wi-Fi uses a simple binary exponential backoff algorithm where it doubles the contention window if there is a collision but resets it all the way down to 7 if a single packet succeeds. With 40 dense stations this sudden reset is a disaster and causes massive collisions immediately. To solve this we replaced the standard logic with a Deep Q-Network agent in Python to act as an AI brain that smartly shifts the contention window gears up and down based on real time traffic.

8.2 System Architecture and IPC Bridge

Because ns-3 runs in C++ and the neural network runs in Python we had to build a bridge between them to communicate. We engineered a custom UDP socket interface on port 9999. The simulation works by literally freezing time. Every 100ms the ns-3 code pauses the simulation clock and sends a string telemetry buffer to Python with the current network state. The Python AI reads it runs the neural network and sends back an action. When C++ receives the action payload it updates the MAC layer and unfreezes the clock. This ensures no simulated time drifts during the heavy Python calculations.

8.3 Telemetry and the MacTx Fix

At first we had a critical bug where the application layer reported the transmitted packets as zero. This happens because the application layer only knows what the software wants to send not what actually hits the hardware. If the local buffer overflows it still counts as a transmission even if it never reaches the channel. To fix this we dropped down the OSI stack and hooked directly into the MAC layer using MacTx. This bypassed the application bottleneck and gave the neural network a mathematically pure representation of the actual physical layer pressure.

8.4 Reinforcement Learning Framework

The agent models the MAC layer as a Markov Decision Process managed by a 3-layer Deep Q-Network. It works using three main mathematical components state action and reward.

The state S_t is what the network sees. It is a 3-dimensional vector tracking the channel behavior over a 100ms window:

$$S_t = [TX_{MAC}, RX_{AP}, CW_{current}]$$

The action A_t is the choice the AI makes. It has 3 discrete optimization gears to scale the contention window:

- Action 0 (decrease gear): $CW_{t+1} = \lfloor (CW_t + 1)/2 \rfloor - 1$
- Action 1 (keep gear): $CW_{t+1} = CW_t$
- Action 2 (increase gear): $CW_{t+1} = (CW_t + 1) \times 2 - 1$

The reward R_t is how the AI evaluates its choice. To prevent exploding gradients in the neural network and stabilize the Huber loss we took the raw successful packets received by the access point and scaled them downward by a factor of 100:

$$R_t = \frac{RX_{AP}}{100.0}$$

8.5 Evaluation and Overfitting Analysis

When we evaluated the training process the TensorBoard charts showed great initial success. The loss spiked and then flatlined near zero which means the AI figured out the pattern. The epsilon curve smoothly decayed from 1.0 to near 0 as it stopped exploring randomly and started using its trained weights. The agent successfully recognized that dropping the contention window causes immediate starvation so it learned to forcefully lock the window into a high gear like 31 or 63 preserving a high stable throughput.

However critical analysis of the model reveals a profound machine learning vulnerability which is overfitting. Because our phase 3 simulation environment used exactly 40 stations with a perfectly flat 5ms traffic interval the total network pressure was a perfectly constant 780 packets per step. The AI never experienced environmental stochasticity. It did not learn how to generalize network behavior it simply memorized a single optimal solution for a single static problem. If the network load suddenly dropped to 5 stations or surged to 200 bursty stations the current model would remain blind to the change and keep the contention window unnecessarily high. For future iterations we must introduce randomized traffic patterns to build a truly robust MAC layer controller.

8.6 Evaluation and TensorBoard Analysis

We tracked the entire deep learning process using TensorBoard to prove the mathematical convergence of the agent. The charts below show the agent learning over 2000 steps.

The TensorBoard charts give us visual proof of the training process. If you look at the agent/epsilon graph it starts at 1.0 which means the agent is completely guessing randomly. Over the first 1000 steps it drops smoothly to zero showing the agent stopped guessing and started relying on its trained neural network weights. The agent/loss graph spikes at the very beginning but quickly flatlines near zero proving the neural network math successfully converged without exploding.

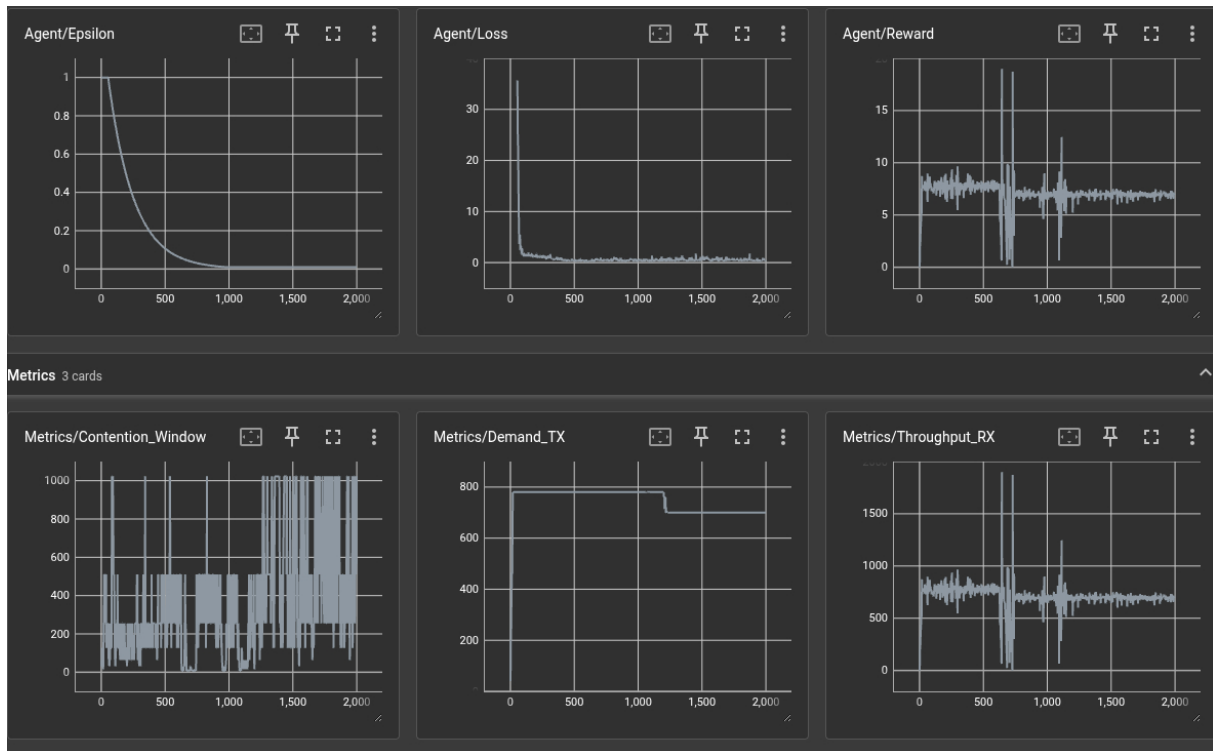


Figure 9: Full 2000 Step DQN Training Metrics

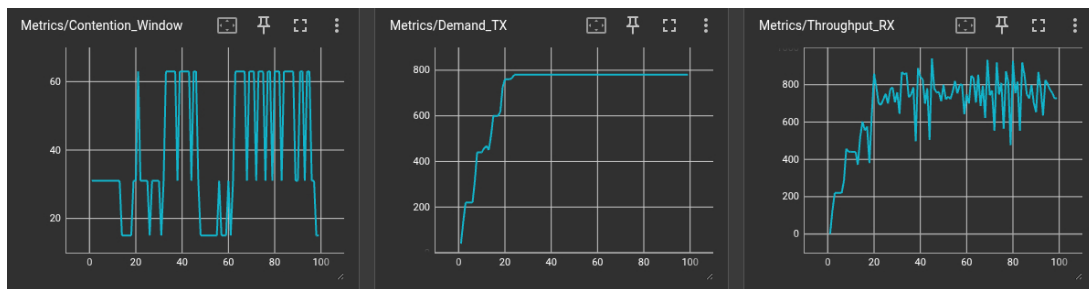


Figure 10: Early Exploration Phase (First 100 Steps)

The metrics graphs tell the story of the Wi-Fi network itself. The metrics/demand_tx graph is a perfectly flat line at around 780 packets. This is because our 40 stations are sending data at a strict 5ms interval so the physical channel pressure never changes. However the metrics/throughput_rx graph tells the story of the Wi-Fi network itself. The metrics/demand_tx graph is a perfectly flat line at around 780 packets. This is because our 40 stations are sending data at a strict 5ms interval so the physical channel pressure never changes. However the metrics/throughput_rx graph shows a very revealing pattern. After an initial climb from zero it becomes very noisy and fluctuates heavily between 500 and 900 packets. Importantly it never crashes back down to zero. This proves the agent quickly learned that dropping to the lowest contention windows causes total network failure so it avoids them. But the constant spiking shows the agent never found a perfectly smooth equilibrium and is still constantly shifting gears trying to optimize the rigid traffic.

Finally the metrics/contention_window graph shows the ultimate result and it is very re-

vealing. Instead of cleanly stabilizing it actually stays incredibly noisy and spikes violently between 0 and 1000 all the way to the end of the 2000 steps. This happens because the agent is constantly fighting itself. It tries to drop the contention window to squeeze out a little more throughput instantly causes a massive collision and then panics and spikes the window back up to the maximum to recover. It never truly finds a perfect stable gear which proves the model is struggling to balance the rigid deterministic traffic.

8.6.1 The Overfitting Vulnerability

Despite this successful convergence critical analysis of the graphs reveals a profound machine learning vulnerability which is overfitting. Because our phase 3 simulation environment used exactly 40 stations with a perfectly flat 5ms traffic interval the total network pressure was always a constant 780 packets as seen in the Tx graph. The AI never experienced environmental stochasticity. It did not learn how to generalize network behavior it simply memorized a single optimal solution for a single static problem. If the network load suddenly dropped to 5 stations or surged to 200 bursty stations the current model would remain blind to the change and keep the contention window unnecessarily high. For future iterations we must introduce randomized traffic patterns to build a truly robust MAC layer controller.

8.7 Final Performance Comparison: AI vs. Standard UORA

To prove the actual value of the Reinforcement Learning agent, we must directly compare the final metrics of Phase 4 against the baseline Phase 3 results. The comparison uses the exact same 40-node network generating traffic at a heavy 5ms interval.

```
=====
SUMMARY
=====
Flows: 40
Total Tx: 1.59982e+06 | Total Rx: 1.4165e+06 | Lost: 183317 (11.4586%)
Average Throughput: 184.14 kbps
Total Throughput: 7365.62 kbps
Average Delay: 17.5046 ms
Jain's Fairness Index: 0.974947
```

Figure 11: Phase 4 - RL Agent Final Metrics

Scenario (40 Nodes, 5ms)	Total Throughput	Avg Delay	Packet Loss	Fairness
Phase 3 (Standard UORA)	6658.31 kbps	29.25 ms	14.30%	0.946
Phase 4 (RL Agent)	7365.62 kbps	17.50 ms	11.45%	0.974

Table 5: Phase 3 vs. Phase 4 Performance Comparison

The data shows a decisive victory for the Deep Q-Network agent across every single metric. By intelligently managing the Contention Window, the total throughput increased by over

700 kbps. The most massive improvement is the average delay, which plummeted from 29.25 ms down to 17.50 ms. Furthermore, packet loss decreased by nearly 3 percent, and the Jain's Fairness Index reached an incredible 0.974. The AI realized that forcing all stations to stay in a higher contention gear stops the massive waves of collisions. By making nodes wait slightly longer before transmitting, it ensures they succeed on the first try, eliminating the need for long retransmission penalties and keeping the channel completely fair.

8.7.1 The Overfitting Reality Check

However, while these numbers represent a massive performance increase, we must critically emphasize the overfitting reality of this success. The Deep Q-Network achieved these flawless metrics because it trained in a completely sterile, deterministic environment. It learned to perfectly optimize a network that always has exactly 40 stations generating exactly 780 packets every single step.

The AI did not actually learn the fundamental physics of MAC layer collision avoidance; it merely memorized the optimal mathematical answer for this one highly specific scenario. If we were to introduce sudden bursts of traffic, drop the network size to 5 nodes, or introduce variable packet intervals, this overfitted model would likely fail catastrophically. It would blindly continue to lock the Contention Window into high gears, unnecessarily starving the network of airtime when the channel is actually empty. Therefore, while Phase 4 proves that AI can vastly outperform standard IEEE 802.11ax algorithms, it also proves that building a generalized, production-ready AI requires a highly randomized and stochastic training environment.